

Distributed Job Scheduler — System Design Document

1. What We're Building

A multi-tenant distributed job scheduler SaaS. Tenants submit jobs (one-time or recurring), the system schedules and executes them reliably, with three deep features:

1. **Multi-tenant quota enforcement** — per-tenant rate limiting and fair-share scheduling
 2. **DAG job dependencies** — jobs with dependency chains, topological execution order
 3. **Live execution log streaming** — real-time WebSocket log tailing per job execution
-

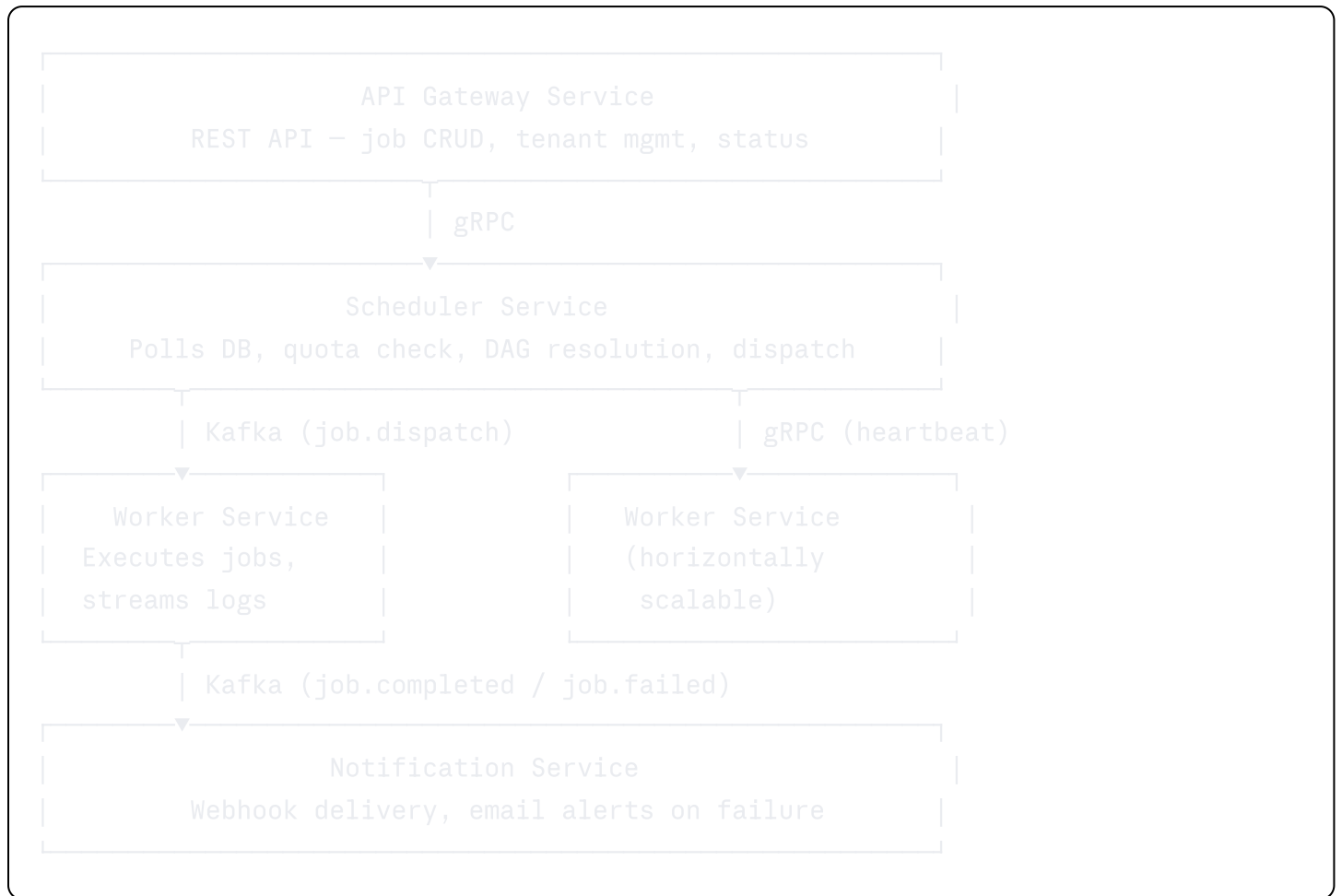
2. Tech Stack

Layer	Technology
Language	Java 21
Framework	Spring Boot 3.3
Inter-service sync	gRPC
Inter-service async	Apache Kafka
Primary DB	PostgreSQL 16
Distributed lock	Redis 7
Real-time	WebSocket (STOMP)
Containerization	Docker + Docker Compose
Deployment	Fly.io / Oracle Cloud Free Tier

Modern Java 21 Features Used

- **Virtual threads** (Project Loom) — worker job execution, handles thousands of concurrent jobs without thread pool tuning
- **Structured concurrency** — clean worker lifecycle, cancel subtasks on parent failure
- **Records** — immutable DTOs, job definitions, Kafka event payloads
- **Sealed classes + pattern matching** — exhaustive job state and failure type modeling
- **Switch expressions** — failure fingerprinting logic

3. Services



3.1 API Gateway Service

- REST endpoints for tenants: submit job, cancel job, query status, manage DAG
- Validates tenant API key, enforces request-level quota at ingestion
- Persists job metadata via outbox pattern
- WebSocket endpoint for live log streaming (proxies from Worker via Kafka)

3.2 Scheduler Service

- Polls `job_executions` table for due jobs (timing wheel approach)
- Checks per-tenant quota before dispatch
- Resolves DAG — only dispatches jobs whose dependencies are `COMPLETED`
- Acquires Redis distributed lock per job before dispatch (prevents duplicate execution)
- Publishes to Kafka `job.dispatch` topic
- Receives Worker heartbeats via gRPC, marks stale leases for re-dispatch

3.3 Worker Service

- Consumes `job.dispatch` Kafka topic

- Executes job (HTTP webhook call to tenant-provided URL with payload)
- Runs each execution in a **virtual thread**
- Streams execution logs to Kafka `job.logs` topic in real-time
- Sends heartbeat to Scheduler via gRPC every 5 seconds
- Reports result to Kafka `job.completed` or `job.failed`
- Implements exponential backoff retry logic per failure type

3.4 Notification Service

- Consumes `job.failed` topic
 - Delivers webhook to tenant-configured URL
 - Sends email alert via SMTP/Resend
 - Retries webhook delivery with backoff (separate retry queue)
-

4. Database Schema

4.1 Tenants

```
sql

CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  api_key VARCHAR(512) UNIQUE NOT NULL,
  quota_per_minute INT NOT NULL DEFAULT 100,
  quota_per_day INT NOT NULL DEFAULT 10000,
  tier VARCHAR(50) NOT NULL DEFAULT 'FREE', -- FREE, PRO, ENTERPRISE
  created_at TIMESTAMPTZ DEFAULT now(),
  is_active BOOLEAN DEFAULT true
);
```

4.2 Jobs

```
sql
```

```

CREATE TABLE jobs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  name VARCHAR(255) NOT NULL,
  description TEXT,
  type VARCHAR(50) NOT NULL, -- ONE_TIME, CRON
  cron_expression VARCHAR(100),
  next_execution_at TIMESTAMPTZ,
  payload JSONB NOT NULL,
  target_url VARCHAR(1024) NOT NULL,
  http_method VARCHAR(10) NOT NULL DEFAULT 'POST',
  headers JSONB,
  timeout_seconds INT NOT NULL DEFAULT 30,
  max_retries INT NOT NULL DEFAULT 3,
  status VARCHAR(50) NOT NULL DEFAULT 'ACTIVE', -- ACTIVE, PAUSED, DELETED
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_jobs_tenant_id ON jobs(tenant_id);
CREATE INDEX idx_jobs_next_execution ON jobs(next_execution_at) WHERE status = 'ACTIVE';

```

4.3 Job Dependencies (DAG)

sql

```

CREATE TABLE job_dependencies (
  job_id UUID NOT NULL REFERENCES jobs(id),
  depends_on_job_id UUID NOT NULL REFERENCES jobs(id),
  PRIMARY KEY (job_id, depends_on_job_id),
  CONSTRAINT no_self_dependency CHECK (job_id != depends_on_job_id)
);

```

4.4 Job Executions

sql

```

CREATE TABLE job_executions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id UUID NOT NULL REFERENCES jobs(id),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  -- PENDING, LEASED, RUNNING, COMPLETED, FAILED, DEAD_LETTER
  scheduled_at TIMESTAMPTZ NOT NULL,
  leased_at TIMESTAMPTZ,
  lease_expires_at TIMESTAMPTZ,
  worker_id VARCHAR(255),
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  attempt INT NOT NULL DEFAULT 1,
  next_retry_at TIMESTAMPTZ,
  failure_type VARCHAR(100),
  -- NETWORK_TIMEOUT, HTTP_ERROR, BUSINESS_LOGIC_ERROR, OOM, UNKNOWN
  error_message TEXT,
  response_code INT,
  response_body TEXT,
  idempotency_key VARCHAR(512) UNIQUE NOT NULL
);

CREATE INDEX idx_executions_status_scheduled ON job_executions(status, scheduled_at)
CREATE INDEX idx_executions_job_id ON job_executions(job_id);
CREATE INDEX idx_executions_tenant_id ON job_executions(tenant_id);
CREATE INDEX idx_executions_lease_expires ON job_executions(lease_expires_at)
  WHERE status = 'LEASED';

```

4.5 Tenant Quota Tracking

```

sql

CREATE TABLE tenant_quota_usage (
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  window_start TIMESTAMPTZ NOT NULL,
  window_type VARCHAR(20) NOT NULL, -- MINUTE, DAY
  job_count INT NOT NULL DEFAULT 0,
  PRIMARY KEY (tenant_id, window_start, window_type)
);

```

4.6 Outbox (for reliable Kafka publishing)

```

sql

```

```
CREATE TABLE outbox_events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  aggregate_id UUID NOT NULL,  
  aggregate_type VARCHAR(100) NOT NULL,  
  event_type VARCHAR(100) NOT NULL,  
  payload JSONB NOT NULL,  
  published BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE INDEX idx_outbox_unpublished ON outbox_events(created_at) WHERE published = false;
```

5. API Design (REST)

Job Management

```
POST    /api/v1/jobs                - submit job  
GET     /api/v1/jobs                - list jobs (paginated, filtered)  
GET     /api/v1/jobs/{id}          - get job details  
PATCH  /api/v1/jobs/{id}          - update job (pause, resume, update cron)  
DELETE  /api/v1/jobs/{id}          - cancel job  
  
POST    /api/v1/jobs/{id}/dependencies - add dependency  
DELETE  /api/v1/jobs/{id}/dependencies/{depId} - remove dependency  
GET     /api/v1/jobs/{id}/dag       - visualize DAG
```

Executions

```
GET     /api/v1/executions          - list executions (filtered by  
status, job, date)  
GET     /api/v1/executions/{id}     - execution detail  
POST    /api/v1/executions/{id}/retry - manual retry  
GET     /api/v1/executions/{id}/logs - paginated log history  
WS      /ws/executions/{id}/logs    - live log stream
```

Tenant

```
POST    /api/v1/tenants             - register tenant  
GET     /api/v1/tenants/me          - tenant profile + quota usage  
GET     /api/v1/tenants/me/stats    - execution stats, success rate
```

6. gRPC Contracts

Scheduler ↔ Worker (Heartbeat + Lease)

```
protobuf

service WorkerRegistry {
  rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse);
  rpc AcquireLease(LeaseRequest) returns (LeaseResponse);
  rpc ReleaseLease(ReleaseRequest) returns (ReleaseResponse);
}

message HeartbeatRequest {
  string worker_id = 1;
  repeated string active_execution_ids = 2;
  int32 available_slots = 3;
}

message HeartbeatResponse {
  repeated string revoked_execution_ids = 1; // scheduler tells worker to stop the
}

message LeaseRequest {
  string execution_id = 1;
  string worker_id = 2;
  int32 lease_duration_seconds = 3;
}
```

7. Kafka Topics

Topic	Producer	Consumer	Key	Description
job.dispatch	Scheduler	Worker	tenant_id	Job ready for execution
job.completed	Worker	Scheduler, Notification	job_id	Execution succeeded
job.failed	Worker	Scheduler, Notification	job_id	Execution failed
job.logs	Worker	API Gateway	execution_id	Real-time log lines
job.dag.unblocked	Scheduler	Scheduler	job_id	Dependency satisfied, job unblocked

Partition by `tenant_id` on `job.dispatch` — ensures per-tenant ordering and natural isolation.

8. Deep Feature 1: Multi-Tenant Quota Enforcement

Design

Every tenant has `quota_per_minute` and `quota_per_day`. Enforced at two layers:

Layer 1 — Ingestion (API Gateway)

- On job submission, check current minute window usage in Redis
- Redis key: `quota:{tenant_id}:minute:{epoch_minute}` with TTL of 2 minutes
- Reject with HTTP 429 if over limit

Layer 2 — Dispatch (Scheduler)

- Before dispatching due jobs, check tenant's remaining quota for the window
- Weighted fair-share: if multiple tenants have due jobs, dispatch proportionally to their tier weight (FREE=1, PRO=5, ENTERPRISE=20)
- Never starve lower-tier tenants — always dispatch at least 1 job per tenant per cycle if due

Weighted Fair-Share Algorithm

```
for each scheduling cycle:
  1. collect all due jobs grouped by tenant
  2. calculate each tenant's weight (tier-based)
  3. total_slots = worker_available_slots
  4. allocate slots proportionally: tenant_slots = (tenant_weight /
total_weight) * total_slots
  5. cap at tenant's remaining quota
  6. dispatch up to tenant_slots jobs per tenant
```

Java 21 Implementation Note

Use `sealed interface QuotaResult` permits `QuotaAllowed`, `QuotaExceeded`, `QuotaTenantNotFound` — pattern match at dispatch site for exhaustive handling.

9. Deep Feature 2: DAG Job Dependencies

Design

Jobs can declare dependencies on other jobs. A job is only dispatched when all its dependencies have `COMPLETED` status in the current execution cycle.


```
On POST /api/v1/jobs/{id}/dependencies:
1. Load full dependency graph for the tenant
2. Temporarily add the new edge
3. Run DFS topological sort (Kahn's algorithm)
4. If cycle detected → reject with 400 + explanation
5. If valid → persist to job_dependencies table
```

Execution Flow

```
When job execution completes (COMPLETED):
1. Worker publishes job.completed event
2. Scheduler consumes event
3. Scheduler queries: SELECT job_id FROM job_dependencies WHERE depends_on_job_id = ?
4. For each dependent job:
  a. Check if ALL its dependencies are COMPLETED in this run
  b. If yes → publish job.dag.unblocked event
  c. Scheduler picks up unblocked event → dispatches job
```

Partial DAG Failure

If a dependency fails, all downstream jobs in that chain move to `BLOCKED_BY_FAILURE` status. Tenant can either:

- Manually retry the failed job (unblocks the chain if it succeeds)
- Cancel the entire DAG chain

Java 21 Implementation Note

Model graph nodes as `record JobNode(UUID jobId, List<UUID> dependencies, JobStatus status)`. Use `SequencedCollection` for topological order result.

10. Deep Feature 3: Live Execution Log Streaming

Design

Workers produce structured log lines to Kafka `job.logs` topic during execution. API Gateway consumes and streams to connected WebSocket clients.

Flow

```
Worker executes job
→ every log line published to Kafka job.logs
```

- API Gateway Kafka consumer picks up log line
- looks up active WebSocket sessions for that execution_id
- pushes via STOMP to /topic/executions/{execution_id}/logs
- client receives real-time

Log Line Structure

```
java

record ExecutionLogLine(
    UUID executionId,
    UUID tenantId,
    Instant timestamp,
    LogLevel level,           // INFO, WARN, ERROR
    String message,
    Map<String, String> metadata // http_status, duration_ms, attempt, etc.
) {}
```

Persistence

Log lines also persisted to `execution_logs` table for historical queries:

```
sql

CREATE TABLE execution_logs (
    id BIGSERIAL PRIMARY KEY,
    execution_id UUID NOT NULL REFERENCES job_executions(id),
    tenant_id UUID NOT NULL,
    logged_at TIMESTAMPTZ NOT NULL,
    level VARCHAR(20) NOT NULL,
    message TEXT NOT NULL,
    metadata JSONB
);

CREATE INDEX idx_logs_execution_id ON execution_logs(execution_id, logged_at);
```

WebSocket Auth

WebSocket connection authenticated via short-lived token (tenant generates via `POST /api/v1/ws-token`). Token carries tenant_id claim. Validated on WS handshake. Worker log lines include tenant_id — API Gateway drops lines that don't match the connected tenant's claim.

11. Job Leasing (Preventing Duplicate Execution)

Critical for at-least-once guarantee without double execution.

Scheduler:

1. SELECT FOR UPDATE SKIP LOCKED on job_executions WHERE status='PENDING' AND scheduled_at <= now()
2. Set status='LEASED', lease_expires_at = now() + 60s, worker_id = assigned_worker
3. Publish to Kafka job.dispatch

Worker:

1. Receives job from Kafka
2. Verifies lease is still valid (not expired, worker_id matches self)
3. If invalid → drops message (another worker already took it)
4. If valid → execute, update status to RUNNING
5. Sends heartbeat every 5s to extend lease

Scheduler (background):

1. Every 30s: find LEASED jobs where lease_expires_at < now() AND no heartbeat
2. Reset to PENDING for re-dispatch

12. Failure Fingerprinting + Smart Retry

Sealed type hierarchy:

```
java

sealed interface FailureType permits
    NetworkTimeout,
    HttpClientError,    // 4xx – don't retry
    HttpServerError,    // 5xx – retry
    WorkerOOM,          // retry on different worker
    BusinessLogicError, // don't retry (job payload issue)
    Unknown

// Retry strategy per type
RetryStrategy strategyFor(FailureType type) = switch (type) {
    case NetworkTimeout t    -> exponentialBackoff(base: 5s, max: 5min, jitter: true)
    case HttpServerError e   -> exponentialBackoff(base: 30s, max: 1hr, jitter: true)
    case HttpClientError e   -> noRetry(); // tenant's endpoint rejected it
    case WorkerOOM o         -> immediateRetry(differentWorker: true);
    case BusinessLogicError b -> noRetry();
    case Unknown u           -> exponentialBackoff(base: 10s, max: 30min, jitter: true)
};
```

After `max_retries` exhausted → status = `DEAD_LETTER`, Notification Service fires alert.

13. Outbox Pattern

Ensures DB write and Kafka publish are atomic — no lost events on crash.

```
Job submitted:
```

```
1. BEGIN TRANSACTION
2. INSERT INTO job_executions (status='PENDING')
3. INSERT INTO outbox_events (event_type='JOB_CREATED', payload=...)
4. COMMIT
```

```
Outbox publisher (background thread, every 100ms):
```

```
1. SELECT * FROM outbox_events WHERE published=false ORDER BY created_at LIMIT 100
2. Publish each to Kafka
3. UPDATE outbox_events SET published=true WHERE id IN (...)
```

Kafka producer set to `acks=all`, idempotent producer enabled. Consumer idempotency via `idempotency_key` unique constraint on `job_executions`.

14. Project Structure (Monorepo)

```
distributed-job-scheduler/
├─ api-service/           # Spring Boot – REST + WebSocket
├─ scheduler-service/     # Spring Boot – polling + dispatch
├─ worker-service/        # Spring Boot – execution + log streaming
├─ notification-service/  # Spring Boot – webhook + email
├─ proto/                 # Shared .proto files
├─ common/                # Shared Java records, sealed types, utils
├─ docker-compose.yml     # Local dev: all services + Kafka + PG + Redis
├─ k8s/                   # Kubernetes manifests (stretch goal)
├─ docs/
│   └─ design.md          # This document
│   └─ adr/               # Architecture Decision Records
```

15. GitHub Kanban — Task Breakdown

Milestone 0: Foundation

- ☐ Init monorepo, parent pom.xml, common module
- ☐ Docker Compose: PostgreSQL, Redis, Kafka, Zookeeper
- ☐ Flyway migrations: all schema tables
- ☐ gRPC proto definitions + code generation setup

- ☐ Shared records: JobEvent, ExecutionLogLine, FailureType sealed hierarchy
- ☐ API key auth filter (Spring Security)

Milestone 1: Core Scheduling (No DAG, No Quota)

- ☐ Job CRUD REST API
- ☐ Cron expression parser + next_execution_at computation
- ☐ Scheduler polling loop (timing wheel approach)
- ☐ Job leasing with SELECT FOR UPDATE SKIP LOCKED
- ☐ Redis distributed lock per execution
- ☐ Kafka producer (job.dispatch)
- ☐ Worker Kafka consumer
- ☐ HTTP job execution (WebClient, virtual threads)
- ☐ Heartbeat gRPC server (Scheduler) + client (Worker)
- ☐ Stale lease recovery background job
- ☐ Outbox publisher
- ☐ Basic execution status API

Milestone 2: Failure Handling

- ☐ Sealed FailureType hierarchy
- ☐ Failure fingerprinting (parse HTTP response, exception type)
- ☐ Smart retry with exponential backoff + jitter
- ☐ Dead letter queue handling
- ☐ Notification Service: webhook delivery
- ☐ Notification Service: email via Resend

Milestone 3: Multi-Tenant Quota

- ☐ Tenant registration + API key generation
- ☐ Redis quota tracking (minute + day windows)
- ☐ Ingestion-level quota check (API Gateway filter)
- ☐ Tier-based weighted fair-share scheduler
- ☐ Quota usage stats API
- ☐ 429 handling with Retry-After header

Milestone 4: DAG Dependencies

- ☐ Job dependency API (add, remove, visualize)
- ☐ Cycle detection on dependency add (Kahn's algorithm)
- ☐ DAG-aware dispatch (check all deps COMPLETED)
- ☐ job.dag.unblocked Kafka topic + consumer
- ☐ Partial failure propagation (BLOCKED_BY_FAILURE status)
- ☐ Manual retry unblocking chain

Milestone 5: Live Log Streaming

- ☐ Worker structured log publisher (Kafka job.logs)
- ☐ execution_logs table + persistence consumer
- ☐ WebSocket STOMP setup (API Gateway)
- ☐ WS token endpoint + auth
- ☐ Kafka consumer → WebSocket session fan-out
- ☐ Historical log query API (paginated)
- ☐ Log level filtering

Milestone 6: Polish + Deploy

- ☐ Prometheus metrics (Micrometer) — job throughput, quota hits, execution latency
- ☐ OpenTelemetry distributed tracing
- ☐ Structured JSON logging (Logback)
- ☐ Dockerfile per service (multi-stage, eclipse-temurin:21-jre-alpine)
- ☐ Fly.io deployment config
- ☐ README with architecture diagram
- ☐ Postman collection

16. Architecture Decisions (ADR Summary)

Decision	Choice	Reason
Job execution model	HTTP webhook	Tenant owns the logic, scheduler is infrastructure
DB polling vs pure Kafka	DB poll + Kafka dispatch	DB is source of truth, Kafka for fan-out
Locking mechanism	Redis + SELECT FOR UPDATE SKIP LOCKED	Redis for speed, DB lock as fallback
Retry coordination	Scheduler-side (not Worker)	Centralized retry state, workers are stateless
Log storage	Kafka → DB (async)	Real-time via Kafka, queryable via DB
Multi-tenancy isolation	Shared DB, tenant_id column	Simpler for portfolio scale, row-level security
gRPC vs REST internal	gRPC	Typed contracts, bidirectional streaming for heartbeat